

1. Student Marks Analyzer

Write a program that accepts marks of 10 students in a list and:

- Calculate the average marks.
- Find the highest and lowest marks.
- Count students who passed and failed.
- Display the grade of each student based on the marks.

2. Frequency of Elements

Write a program to accept a list of integers and create a dictionary containing the **frequency of each element**.

Example:

Input: [2, 3, 2, 5, 3, 2]

Output: {2: 3, 3: 2, 5: 1}

3. Separate Even and Odd Numbers

Write a program that accepts a list of integers and creates two separate lists:

- One containing even numbers
- One containing odd numbers

Also display the sum of numbers in each list.

4. Remove Duplicate Elements

Write a program to accept a list containing duplicate values and create a new list containing only unique elements **without using set()**.

5. Second Highest Number

Write a program to find the **second highest unique number** from a list without using the built-in **sort()** or **sorted()** functions.

6. Employee Salary Analyzer

Create a dictionary containing employee names and salaries.

Write a program to:

- Display employees earning more than ₹50,000.
- Calculate the average salary.

- Find the highest-paid employee.
- Count employees earning below the average salary.

7. Word Frequency Counter

Accept a sentence from the user and create a dictionary containing the frequency of each word.

Ignore differences between uppercase and lowercase words.

Example:

Input: "Python is easy and Python is powerful"

Output:

```
{  
    "python": 2,  
    "is": 2,  
    "easy": 1,  
    "and": 1,  
    "powerful": 1  
}
```

8. Common Elements Between Lists

Accept two lists from the user and find:

- Common elements
- Elements present only in the first list
- Elements present only in the second list
- All unique elements from both lists

Do not use built-in set operations.

9. Number Classification

Accept a list of numbers and classify each number as:

- Positive even
- Positive odd
- Negative even
- Negative odd
- Zero

Store the results in an appropriate dictionary.

10. Shopping Cart Program

Create a shopping cart using a dictionary where:

- Key = product name
- Value = price

Allow the user to add multiple products and quantities.

Calculate:

- Total bill
- Discount based on bill amount
- Final payable amount

11. Find Duplicate Values

Write a program to find all duplicate elements in a list and store them in a separate list.

Example:

Input: [10, 20, 30, 20, 40, 10, 50]

Output: [10, 20]

12. Character Analyzer

Accept a string and create a dictionary containing:

- Number of uppercase characters
- Number of lowercase characters
- Number of digits
- Number of special characters
- Number of spaces

13. Student Database

Create a dictionary containing student names and their marks in three subjects.

Example:

```
students = {  
    "Aman": [78, 85, 91],  
    "Riya": [88, 76, 95],  
    "Karan": [65, 72, 69]  
}
```

Write a program to:

- Calculate total and percentage of every student.
- Assign grades.
- Find the class topper.
- Find the class average.

14. Find Missing Number

Given a list containing numbers from 1 to n with one number missing, find the missing number using loops.

Example:

Input: [1, 2, 3, 5, 6]

Output: 4

15. Product Inventory

Create an inventory dictionary containing product names and quantities.

Write a program that:

- Displays products with quantity below 10.
- Updates product quantities.
- Adds new products.
- Removes products whose quantity becomes zero.
- Displays the total number of products available.

16. Anagram Checker

Write a program that accepts two strings and checks whether they are **anagrams** of each other.

Example:

Input: "listen"

Input: "silent"

Output: Anagram

Do not use external libraries.

17. Password Strength Checker

Write a program that accepts a password and checks whether it satisfies all of the following:

- At least 8 characters
- Contains uppercase character
- Contains lowercase character
- Contains a digit
- Contains a special character

Display the missing requirements if the password is invalid.

18. Sales Data Analyzer

Given sales data in a dictionary:

```
sales = {  
    "January": 45000,  
    "February": 52000,  
    "March": 48000,  
    "April": 61000,  
    "May": 58000  
}
```

Write a program to:

- Find total sales.
- Find average sales.
- Find the month with highest sales.
- Find the month with lowest sales.
- Display months where sales exceeded the average.

19. List Rotation

Write a program to rotate a list to the right by k positions using loops.

Example:

Input: [1, 2, 3, 4, 5]
 $k = 2$

Output: [4, 5, 1, 2, 3]

20. Matrix Row and Column Analysis

Accept a matrix from the user using a nested list.

Write a program to:

- Calculate the sum of each row.
- Calculate the sum of each column.
- Find the largest element.
- Find the smallest element.
- Calculate the sum of the main diagonal.

21. Unique Words from Multiple Sentences

Accept 5 sentences from the user and:

- Extract all words.
- Find unique words.
- Find words appearing in every sentence.
- Find the most frequently occurring word.

Use appropriate Python data structures.

22. Banking Transaction Analyzer

Given a list of transactions:

```
transactions = [  
    ("deposit", 5000),  
    ("withdraw", 2000),  
    ("deposit", 3000),  
    ("withdraw", 1000)  
]
```

Write a program to:

- Calculate the final balance.
- Calculate total deposits.
- Calculate total withdrawals.
- Find the largest transaction.
- Count the number of deposits and withdrawals.
- Display whether the account went below zero at any point.

23. Department-wise Employee Analysis

Create a list of dictionaries:

```
employees = [  
    {"name": "Aman", "department": "IT", "salary": 60000},  
    {"name": "Riya", "department": "HR", "salary": 50000},  
    {"name": "Karan", "department": "IT", "salary": 75000}  
]
```

Write a program to:

- Find the average salary of each department.
- Find the highest-paid employee.
- Count employees in each department.
- Display employees earning above ₹60,000.

24. Movie Rating Analyzer

Create a dictionary containing movie names and ratings.

Write a program to:

- Display movies rated above 8.
- Find the highest-rated movie.
- Find the average rating.
- Count movies in different rating categories:

- Below 5
- 5–7
- 7–8
- Above 8

25. Mini Quiz Application

Create a quiz using a dictionary where questions are keys and correct answers are values.

The program should:

- Ask questions one by one.
- Accept answers from the user.
- Calculate the score.
- Display correct and incorrect answers.
- Display the final percentage.
- Assign a performance level based on the score.